Omniveyor: An Assembled Logistics Sorting System Powered by Reinforcement Learning

Mingrui Yin, Hao Zhang, Chenxin Cai, Meiyan Liang, and Jie Liu, *Fellow, IEEE*

***Abstract*— To improve logistics efficiency, smart logistics sorting is an inevitable trend in logistics development. Existing smart logistics sorting systems suffer from high construction costs or limited scalability. To solve these problems, we design a brand-new two-dimensional conveyor system called Omniveyor, which transports and sorts high-density packages within a limited space. It is assembled from multiple repetitive square conveyor modules, achieving the goal of cost-effectiveness and easy maintenance. To realize automatic sorting, we model the planning problem on Omniveyor and propose a scheduling strategy named MMPPO by reinforcement learning. Unlike traditional path planning, MMPPO assigns actions to modules rather than packages, which reduces scheduling overhead in high-throughput scenarios. Furthermore, we develop a simulation environment to inspect the effectiveness of our method, which solves an intractable package-following problem that has plagued simulation implementation in this field. Experimental results show that MMPPO outperforms baselines in terms of throughput and overall consumption at high densities. Besides, we implement a physical prototype of Omniveyor to validate its feasibility.**

***Note to Practitioners*—The motivation of the paper is to solve the sorting problems in the logistics system. Existing logistics sorting systems often have problems such as low sorting efficiency and high costs, making them unsuitable for small-sized warehouses. In this paper, we present a modular 2D desktop logistics system that is both scalable and efficient for sorting. We mathematically describe the platform package transportation process and propose an algorithm that addresses scheduling and planning problems, capable of continuous planning under pipeline input. Preliminary simulation tests indicate that our approach is feasible, and we have also built a small-scale prototype. In future research, we will further expand the scale of the platform and conduct research.**

***Index Terms*— Logistics sorting system, omnidirectional conveyor, deep reinforcement learning, real-time scheduling.**

Received 29 September 2024; revised 20 December 2024; accepted 5 February 2025. Date of publication 10 February 2025; date of current version 16 April 2025. This article was recommended for publication by Associate Editor N. Frigerio and Editor K. Liu upon evaluation of the reviewers' comments. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0110900, in part by the Key Research and Development Program of Heilongjiang Province under Grant 2022ZX01A22, in part by the National Natural Science Foundation of China under Grant 61972114 and Grant 62106061, in part by the Project of Laboratory of Advanced Agricultural Sciences of Heilongjiang Province under Grant ZY04JD05-010, and in part by the National Natural Science Foundation of Heilongjiang Province under Grant YQ2019F007. *(Corresponding author: Hao Zhang.)*

Mingrui Yin, Hao Zhang, Chenxin Cai, and Meiyan Liang are with the Faculty of Computing, Harbin Institute of Technology, Harbin 150001, China, and also with the National Key Laboratory of Smart Farm Technologies and Systems, Harbin 150001, China (e-mail: zhh1000@hit.edu.cn).

Jie Liu is with the International Research Institute for Artificial Intelligence, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China, and also with the National Key Laboratory of Smart Farm Technologies and Systems, Harbin 150001, China.

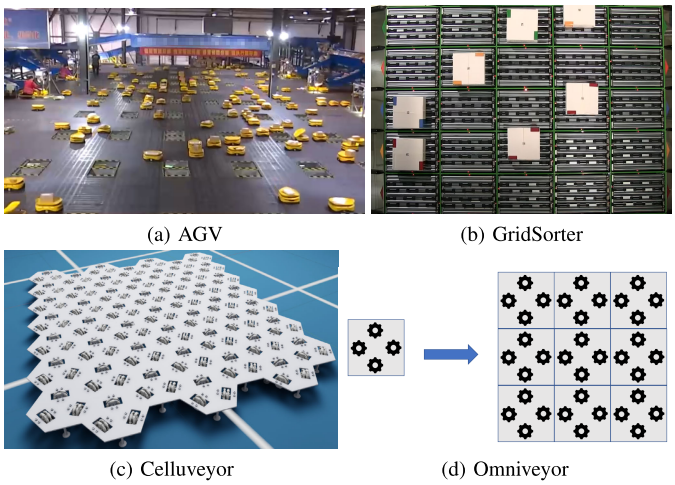
Digital Object Identifier 10.1109/TASE.2025.3540511

I. Introduction

LOGISTICS runs through the production and distribution processes of various industries. To improve logistics efficiency, packages will converge at the sorting centers to be regrouped according to the destinations. In each sorting center, the logistics sorting system is the core part. It persistently classifies incoming packages to different exits depending on where they need to be sent [1]. Traditional sorting systems often rely on manual sorting or conveyor belts. They suffer from inefficiencies, poor durability, and lack of flexibility, failing to meet the demands of large-scale, time-sensitive deliveries [2]. With the rapid growth of online shopping and consumer upgrading, package sorting tends to be small-amount, multi-batch, and high-frequency, which makes it more difficult for traditional logistics sorting systems to meet the current demand. Constructing the smart logistics sorting system becomes the core of modern smart logistics centers [3].

An ideal smart logistics sorting system should provide continuous, efficient performance with minimal errors and the lowest possible cost. Two promising smart sorting systems for logistics applications are Automatic Guided Vehicle (AGV) and two-dimensional Surface Conveyor System (SCS). AGVs are used for horizontal movement of materials (Fig. 1(a)). Despite with high flexibility, it requires high construction and maintenance costs, which limits its applicability [4]. In contrast, SCS is more suitable for sorting and transporting in a densely packed and confined space, especially in small warehouse scenarios. It is usually constructed by concatenating multiple replaceable units, which offers the advantages of flexibility, robustness, and compactness. Fig. 1(b) and Fig. 1(c) show two typical types of SCS. GridSorter [5] (Fig. 1(b)) is optimized for sorting but features a complex mechanical design. While Celluveyor [6] (Fig. 1(c)) uses hexagonal transfer within a lower construction cost. However, its hexagonal structure is incompatible with standard square packages, complicating control.

Besides platform structure, scheduling is also an important part of smart sorting systems, which transmits and sorts packages automatically by leveraging the features of the platform. On SCS, most researches focus on path planning of individual packages [7], [8], which often results in frequent package collisions, particularly in high-density scenarios. More seriously, package collisions may cause deadlocks among packages, which leads to the failure of planning. Krühn et al. [9] and Julia et al. [10] try to solve the collision problem, but the methods are limited to tiny-scale scenarios. On the other hand, the path-finding problem has been widely investigated in AGV [11]. Deep reinforcement learning is regarded as an



(a) AGV (b) GridSorter

(c) Celluveyor (d) Omniveyor

Fig. 1. Overview of different kinds of smart logistics sorting systems. (b), (c) and (d) are two-dimensional Surface Conveyor System.

effective algorithm [12]. However, the quantity of robots in the AGV setting remains constant, rendering these algorithms inadequate for scenarios with varying package numbers in SCS.

In this paper, we introduce Omniveyor, an assembled logistic sorting system based on the concept of SCS(Fig. 1(b)). For the hardware implementation, we design a new square conveyor module to build the platform, achieving the goal of cost-effectiveness and easy maintenance. The platform offers high flexibility in package movement, with each module providing four directional options. Based on the structure, we formulate the scheduling model to minimize the cost of packages on the platform, and employ Maskable Proximal Policy Optimization to dynamically adjust to changes in package quantity in real-time. To adapt to the frequent changes in package quantity, we trade each module rather than packages as a controlling target. We also design a simulation environment to optimize the operational efficiency of the platform. In summary, our contributions are as follows:

1) We design a new SCS named Omniveyor for high-density package sorting on a compact, scalable, and modular platform. Omniveyor is adaptable, and capable of efficiently handling packages that continuously enter the system, which addresses some of the challenges found in current sorting systems under similar conditions.
2) We formulate the sorting goals and conditions of Omniveyor, and propose a module-based scheduling algorithm, MMPPO, specifically designed for the Omniveyor system. Unlike traditional approaches that treat the package as the unit of operation, our algorithm treats each module as the operational unit. This allows the system to better adapt to real-time changes in package quantities, thereby significantly enhancing sorting efficiency.
3) We develop a simulation environment based on Omniveyor.[1] Our environment effectively handles the package-following problem without significant performance loss, providing a reliable platform for testing sorting algorithms.
4) Our experimental results indicate that the MMPPO algorithm demonstrates excellent performance on the Omniveyor platform, with the square-shaped SCS outperforming the hexagonal configuration. In addition to simulations, we construct a small-scale physical system to validate the feasibility of our approach, showcasing its practical applicability.

The paper is structured as follows. Section II briefly introduces relevant works. In Section III, we describe the structure of Omniveyor, and formulate the planning goal and conditions, which is an NP-hard problem. In Section V, we introduce the reinforcement learning method to solve the planning problem efficiently, and the collision solution in simulation. In section VI we show the experimental results and system structure. At last, we summarize the paper in Section VII.

II. Related Work

A. The Platform for Smart Logistics Sorting System

In contemporary logistics sorting systems, the establishment of automated equipment is crucial. AGV and SCS are two main types of sorting system carriers. AGV is a type of portable robot that can be navigated in two ways. One is by following wires marked on the floor, but this method requires modifications of the warehouse [13]. The other is by using visual cameras or lasers for navigation, which requires an expensive initial investment for each vehicle [3]. Additionally, over time, the production and maintenance costs of AGVs will gradually increase [14].

As for SCS, it is a modular logistics sorting system composed of multiple modules with identical mechanical structures and control logic [15]. Modularity reduces construction costs and simplifies maintenance. Unlike traditional conveyor belts, SCS allows packages to be transferred in multiple directions, enabling the creation of various paths from entrance to exit [16]. These paths can overlap and intersect, maximizing platform utilization and improving sorting efficiency, especially on larger scales [2].

Several SCS variants have been developed. The Celluveyor system, characterized by its hexagonal transport modules, allows packages to move in any direction due to independent wheel control, though its design may complicate control when handling standard square packages [6], [7]. The GridSorter represents a quadrilateral grid system with decentralized control, allowing packages to enter and exit from any side. Additionally, scholars have proposed square omni-wheeled cellular conveyors for enhanced control, though such designs often involve higher costs [5], [17]. Consequently, we propose a novel SCS platform to facilitate high-density package sorting, characterized by a compact, scalable, and modular architecture.

B. Control Algorithms for Smart Logistics Sorting Systems

Effective control algorithms are essential to complement the hardware of smart sorting systems. At present, the research

[1]We release our code at https://github.com/mingrui4/Module-based-MPPO-Env

on AGV and related multi-agent robots has been relatively in-depth, mainly focusing on task allocation and path planning. Mathematical models are often used to describe path planning problems in AGV sorting systems [18], [19]. At the same time, other researchers have proposed evolutionary algorithms such as cultural-PSO, Bayesian optimization, and generalized permanent labeling algorithm to address these challenges [20], [21], [22].

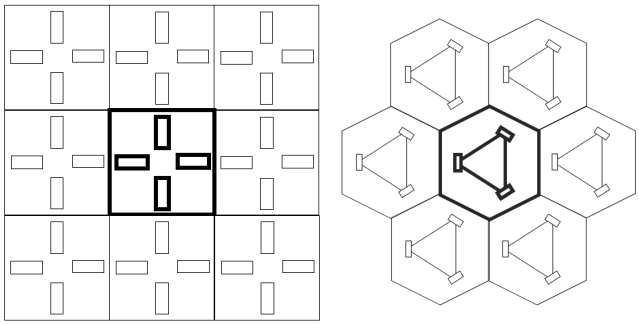
However, control algorithms for SCS still have several deficiencies. On cellular conveyors, considering the uniqueness of packages and controllers separation, scholars tend to focus on the control of distributed systems and path planning for packages. Some researchers still confine their focus to the overall path planning of the packages themselves, without considering packages and platforms as an integrated whole [7], [8]. Others have proposed a decentralized self-organizing control method for modules based on cellular automation, which prevents deadlocks in packages by allowing modules to reserve routes [9], [10]. However, these algorithms have not been thoroughly validated in high-density environments, limiting their applicability in more complex systems.

C. Pathfinding Problem

One approach to solving the control problem is to transform it into a path-planning problem. Pathfinding problem involves planning the collision-free paths between start locations and target locations [23]. It has a wide range of real-world applications such as large-scale warehouse management [24] and multi-robot systems [25], [26]. When evaluating the solutions, researchers usually focus on three main objectives: minimizing the makespan (the last arrival time), minimizing the total arrival time, and minimizing the total distance [27].

Heuristic algorithms like conflict-based search (CBS) [28] are widely used in pathfinding but struggle to scale to large platforms and have limitations in deployment. These algorithms are primarily designed to address "one-shot" problems, where both the initial and goal configurations are fixed, the number of start-goal pairs matches the number of robots, and each robot has a unique destination [29]. In contrast, for dynamic and more complex environments that favor lifelong multi-agent path finding (MAPF), recent works tend to favor the use of reinforcement learning methods [30]. Deep Reinforcement Learning technology enables safe and accurate planning whether or not comprehensive information about the workspace is obtained [31]. Proximal Policy Optimization (PPO), Maskable PPO and their extended algorithms are widely used in pathfinding algorithms [32], [33]. In addition, hybrid approaches such as Pathfinding via Reinforcement and Imitation Multi-agent Learning (PRIMAL) [34] have also been proposed.

However, these algorithms often struggle to adapt effectively to SCS environments due to the separation of packages and controllers. Furthermore, our platform is an assembled platform that can continuously transport packages. This planning method aimed at packages results in changes in the number of agents at any given moment, yet current technology for the fluctuations in agent count remains immature [35]. To address this problem, we propose a centralized control algorithm based on Maskable PPO as a solution.



(a) Square-shaped arrangement (b) Hexagon-shaped arrangement

Fig. 2. The layout of square and hexagonal modules, rectangular grids within each module indicating the positions of wheels.

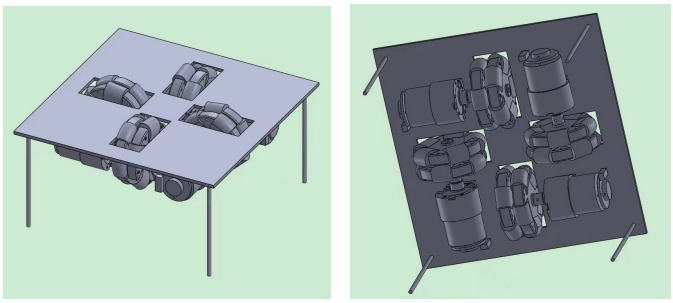


Fig. 3. The "+" shape layout of motors and omni-wheels of our platform.

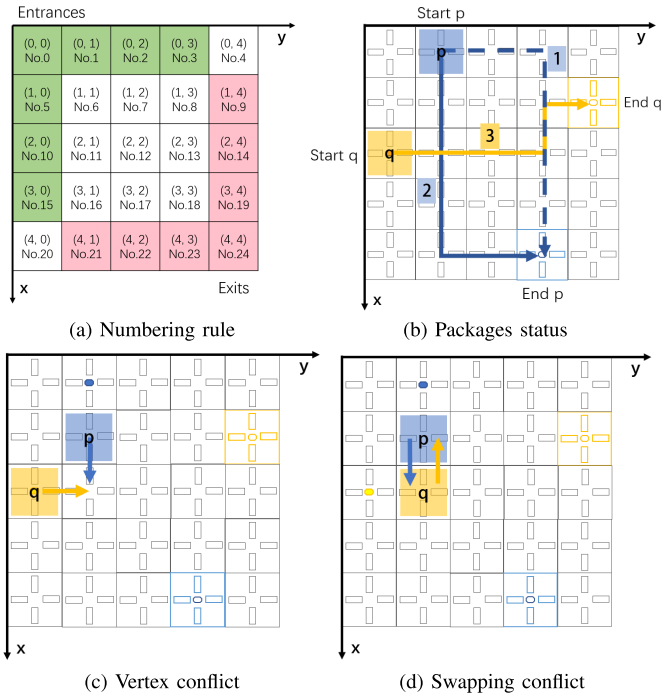
III. Omniveyor Structure and Problem Formulation

We first outline the characteristics and necessity of SCS. Subsequently, we introduce the structure of our proposed system Omniveyor, and explain how it transports packages on the platform. Then, we mathematically model the control and planning problem for Omniveyor and analyze the complexity.

A. The Structure of Omniveyor

We propose an SCS named Omniveyor, which is constructed by the same modules with uniformly geometric shapes. It ensures that all modules can execute omnidirectional transportation of packages in parallel. Therefore, it is critical to design the module, including the geometric shape and how to move packages on it. Fig. 2 shows the common shapes used in SCS, including square (Fig. 2(a)) and hexagon (Fig. 2(b)). Each graphical unit represents a module. In practical testing, we observe that the shapes of hexagonal modules and the packages do not match, causing the package to not align completely with the modules during movement, thus increasing the complexity of path planning. we choose the square module as the basic unit of Omniveyor to control packages precisely. An experimental comparison between the square and hexagonal modules is in Section V-C. An in-depth analysis of the advantages of the square module is in Appendix II.

In Fig. 2, the rectangular grids within each module represent the positions of the wheels responsible for moving packages across the module. To provide the capability of moving packages in four particular directions, we arrange



(a) Numbering rule (b) Packages status

(c) Vertex conflict (d) Swapping conflict

Fig. 4. Platform modeling and collision situations. (a) describe the rule of numbering the grids and one of the layouts of entrances and exits. (b) shows the potential paths from entrances to exits. (c) and (d) show two types of conflicts between the blue package (Box p) and the yellow one (Box q).

the wheels on the module in a "+" shape (Fig. 3), each of which is driven by a motor on the back of the module. The controller simultaneously controls two wheels on the same line to rotate in the same direction, propelling the package on the module to move upwards, downwards, leftwards, or rightwards following the scheduling scheme. When multiple modules are assembled into a platform, packages can be transported to designated locations by controlling the motors. The corresponding scheduling algorithm will be described later.

B. Omniveyor Platform Modeling

To facilitate the description of the functions of Omniveyor, we treat the entire platform as a grid-based 2D workspace where each module is a grid, and address each grid by its position in the platform. We define the address using two-dimensional grid coordinates (x, y) and the serial number (No.). The origin of the coordinates is the top left corner of the grid as (0, 0) and No.0. Subsequently, numbering continues from left to right and top to bottom. Fig. 4(a) illustrates the numbering rules in a 5×5 platform.

In Omniveyor, each grid on the edge can be used as an exit or entrance. To better adapt to the needs of industrial scenarios, we assume that the top and left edges of the platform are the entrances of packages, represented by the green parts in Fig. 4(a). The lower and right edges are the exits shown in red. Note that the modules at the top right and bottom left corners are not used as entrances or exits. Packages enter the platform from the entrance at a predetermined speed but exit at an infinite speed, which means once a given package reaches the exit, it will be removed from the system immediately and will not interfere with the movement of other items.

To simplify the description of the problem, we put forward the following constraints:

- The acceleration and deceleration during package transportation are disregarded.
- Each package strictly follows the control instructions to reach its designated location.
- The size of each package can completely overlap each module.
- Time is discretized into multiple timeslots. In one timeslot, each module selects one transportation operation or remains stationary.

To ease of discussion, failure of the platform is not considered, since the module is simple in structure and easy to replace.

Based on the above design, we describe the path of the package, which starts from one entrance (starting point) to one exit (endpoint). Fig. 4(b) illustrates the status of two packages named Box p and Box q. Each package has a specific starting point and endpoint. Our objective is to control the modules to transport both boxes from their respective starting points to their destinations. For each box, several paths are available from the starting point to the endpoint. Fig. 4(b) shows two paths of Box p and one path of Box q. The paths of different packages may intersect and even overlap, leading to collisions during the transportation process under inappropriate scheduling. For instance, they may collide if Box p follows Path 1 and Box q follows Path 3. Specifically, two types of collisions are mainly taken into account. A *vertex conflict* happens if multiple packages move to the same module at the same time (Fig. 4(c)). A *swapping conflict* happens when two neighboring packages are scheduled to move to the locations of each other (Fig. 4(d)). These collisions lead to the failure of package transportation. It is essential to avoid conflicts in the planning.

C. Problem Formulation

The ideal scheduling method satisfies the following requirements:

- All packages are transmitted correctly from entrances to exits.
- All conflicts should be avoided.
- The *makespan* (i.e., the maximum completion time of package transferring from the entrance to the exit for all packages) is as short as possible.
- The power consumption for transferring packages is as low as possible.

The first requirement ensures the functional correctness of Omniveyor. The second one ensures the unattended running of Omniveyor. The third one maximizes the efficiency of the platform, and the last one minimizes the energy consumption of Omniveyor. The energy of Omniveyor is mainly consumed by moving packages. To simplify the analysis, we assume the moving actions of one module consume the same amount of energy, regardless of the direction of movement and the difference of packages. The energy consumption is negligible when packages remain stationary. Therefore, the energy consumption of Omniveyor can be quantified by the total number of actions of all modules.

Based on these requirements, we formulate the problem of finding the optimal scheduling as

$$\min_{a} \quad \beta_1 \sum_{k\in K}\sum_{t\in T} c_{k,t} + \beta_2 \tau \tag{1}$$

$$\text{s.t.} \quad z^0_{b,\text{sp}_b} = 1, \quad \forall b \in B \tag{2}$$

$$z^t_{b,\text{ep}_b} = 1, \quad \forall b \in B, \exists t \in T \tag{3}$$

$$\sum_{t=0}^{\tau}\sum_{b\in B} z^t_{b,\text{ep}_b} = |K|, \tag{4}$$

$$\sum_{k\in K} z^t_{b,k} \leqslant 1, \quad \forall b \in B, t \in T \tag{5}$$

$$\sum_{b\in B} z^t_{b,k} \leqslant 1, \quad \forall k \in K, t \in T \tag{6}$$

$$z^t_{p,i} \cdot z^t_{q,j} + z^{t+1}_{p,j} \cdot z^{t+1}_{q,i} \leqslant 1, \quad \forall p, \forall q \in B, i, j \in K, t \in T \tag{7}$$

$$\boldsymbol{\zeta}_{t+1} = \sum_{a=0}^{4} (\boldsymbol{\zeta}_t \times (\boldsymbol{\alpha}_t \overset{\circ}{=} a)) \cdot \mathbf{M}_a, \forall t \in T \tag{8}$$

$$c_{k,t} = \begin{cases} 1 & a_{k,t} = 1, 2, 3, 4 \\ 0 & a_{k,t} = 0, \end{cases} \quad \forall t \in T, k \in K \tag{9}$$

$$z^t_{b,t} \in \{0, 1\}, \quad \forall b \in B, k \in K, t \in T \tag{10}$$

$$c_{k,t} \in \{0, 1\}, \quad \forall k \in K, t \in T \tag{11}$$

All symbols are described in Table I. For the convenience of understanding the problem, We explain the above equations as follows:

1) **Objective.** Eq. (1) denotes that our objective is to minimize energy consumption and makespan τ in a given period of time.
2) **Makespan.** Eq. (2) ensures that all packages should be positioned at their starting points at time 0. Eq. (3) ensures that all packages must reach their endpoints before the end time. Eq. (4) defines τ, which equals the time t that all packages have arrived at their endpoints.
3) **Collision.** Eq. (5) means that each package is on one module, or not on the platform in each timeslot. Eq. (6) dictates that each module holds only one package or none in each timeslot. These two equations avoid vertex conflict among all packages. Eq. (7) avoids swapping conflict between two packages.
4) **Controlling.** Eq. (8) shows the relationship between the positions of packages and the actions of modules. $\boldsymbol{\alpha}_t \overset{\circ}{=} a$ results in a binary vector with the same dimension of $\boldsymbol{\alpha}_t$, where each element indicates whether the corresponding element of $\boldsymbol{\alpha}$ is equal to a. Eq. (9) denotes the relationship between actions and cost. Eq. (10) and Eq.(11) are the integer restrictions of the binary variables z and c.

D. Complexity Analysis

In the planning and scheduling problem on Omniveyor, our optimization goals are minimizing the makespan and the cost of all modules. Although we treat the platform rather than the package as the agent, these parameters can be decomposed into each package. Our problem can be seen as the MAPF problem, which is an NP-hard problem, involving finding collision-free paths for multiple agents while optimizing criteria including makespan and total distance [27], [36], [37]. Concretely, assuming there is a set of packages, $B = \{b_1, \ldots, b_n\}$, moving in a connected and undirected graph $G = (V, E)$, with vertex set $V = v_i$ and edge set $E = (v_i, v_j)$. The location of each package, $b^i_t \in V$, is defined at discrete time $t \in T$. Packages can either move to adjacent nodes or stay in their current location. Thus, our problem can be transformed into a traditional MAPF problem on a grid map, in which

$$|z^t_{b,k} - z^{t+1}_{b,k}| = c_{k,t}, \quad \forall b \in B, t \in T, k \in K, \tag{12}$$

and the Objective (1) can be set as

$$min \quad \alpha_1 \sum_{b\in B}\sum_{k\in K}\sum_{t\in T} |z^t_{b,k} - z^{t+1}_{b,k}| + \alpha_2 t_\tau, \tag{13}$$

with Constrains Eq. (2) to Eq. (7) and Eq. (10).

Based on the above analysis, finding the optimal scheduling method is an NP-hard problem. Furthermore, it is a nonlinear problem. It is difficult to solve it with traditional solvers.

TABLE I
NOTATIONS USED IN THIS PAPER

Symbol	Description
i, j, k	the index of modules
p, q, b	the index of packages
t	the index of time steps
a	the index of actions
τ	the makespan
B	the set of packages
K	the set of modules, $\|K\|$ means the number of modules.
T	the set of time steps, $T = \{1, 2, \ldots, t_{\max}\}$, $t_{\max}$ indicates the time required for traversing the wrapping platform, larger than τ.
sp_b	the starting point of package, $b \in B$.
ep_b	the end point of package, $b \in B$.
$z^t_{b,k}$	a binary variable, equals 1 if package b is at module k at time t, otherwise 0.
$a_{k,t}$	an int variable, equals 0,1,2,3,4 if module k has action as stay, push up, down, left, right.
$c_{k,t}$	a binary variable, equals 1 if the module k has an action at time t, otherwise 0.
$\boldsymbol{\zeta}_t$	a vector with $\|K\|$ dimension, when there is a package located on module k at time t, $\boldsymbol{\zeta}_{t,k}$ will be 1.
$\boldsymbol{\alpha}_t$	a vector with $\|K\|$ dimension, includes the actions of all modules at time t.
$\mathbf{M}_a$	a state-transition matrix with $\|K\|\times\|K\|$ dimension, $\mathbf{M}_a$ transfer state vector $\mathbf{Z}_t$ at time t to be $\mathbf{Z}_{t+1}$ at $t+1$ with action a.

IV. Reinforcement Learning-Based Scheduling Method

In this section, we introduce how to find a nearly optimal scheduling method for Omniveyor based on reinforcement learning strategies. Maskable PPO [38] is a proper candidate method, which is widely used in pathfinding algorithms. However, unlike the traditional usage of Maskable PPO which is based on packages, we propose a modular version (MMPPO), which makes the algorithm complexity independent of the number of packages, especially suitable for high-load situations. Since we can access global information about all packages and modules on Omniveyor, MMPPO uses a centralized way to control the motors of all modules.

A. Environment Creation

We consider the entire platform as a Markov decision process (MDP). A standard MDP tuple consists of (S, A, R, S'), where S is the state, A is the action, R is the reward, and S' is the next state. In this section, we will introduce the observation space, action space, and reward structure of the MDP in detail.

1) Observation Space: In our discrete grid-based platform with global information, the planner has access to complete information about all packages and modules. We divide the available information into two different channels in each timeslot. One contains the real-time locations of packages, and the other contains the destinations of packages. Each of both can be modeled as a two-dimensional $m \times n$ matrix, where m and n are the dimensions of the platform. Following the above notations, $|K| = m \times n$ corresponds to the total number of modules.

Given that the primary task of the planner is to ensure the delivery of all packages to their designated destinations, to simplify the observation space, we categorize packages based on their destination. For example, if different packages with the same destination are located at (x_1, y_1) and (x_2, y_2) at time t, we assign them the same number in the first matrix. Simultaneously, in the second matrix, we assign the destination (a, b) also with the same number. This approach enables the planner to identify and group packages with similar behavioral patterns and destinations, reducing the observation space.

2) Action Space: In the grid world, actions are represented as a one-dimensional discrete array with a length of $m \times n$. The planner takes discontinuous actions to move the package to an adjacent module along up, down, left, and right directions or remain stationary, corresponding to values 1, 2, 3, 4, and 0 respectively (Fig. 5).

3) Reward Structure: The reward function in our system follows a typical pattern observed in grid worlds. At each time step, the package will be punished if it does not reach its endpoint.

a) Motivating to deliver package: The system will receive a sizeable reward when all packages are successfully delivered to the specified destinations. To enhance the throughput of the platform, necessitating swift delivery of packages, we penalize stationary modules more heavily than those in motion. To expedite training, we slightly increase penalties for movements away from the endpoint compared to movements toward it.

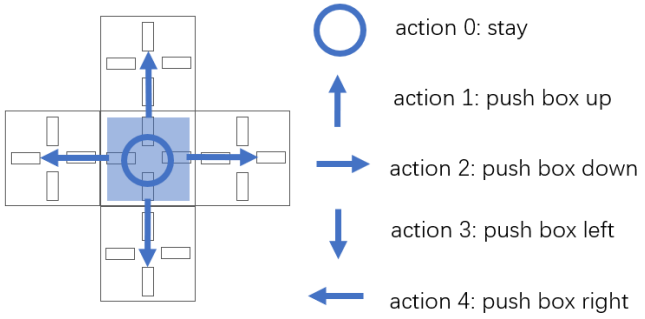


Fig. 5. We use numbers to indicate actions for each module: 0 for stay and 1,2,3,4 for push up, down, left, and right respectively.

b) Solving collision: In the description in the previous section, we did not filter out collision problems, so we need to let the package avoid collisions. When a collision occurs, we will give a relatively high penalty. Based on the analysis, we list the specific values of rewards in Table II using in our method.

B. Module-Based Maskable PPO

Although the density of packages can reach a high level on our platform compared to traditional conveyor belts, the distribution of packages remains relatively sparse across individual modules. For one module, its action is considered as an invalid operation when it is not carrying a package. Typically, negative rewards are assigned to such actions to discourage them. However, in a sparse environment, it is challenging for the agent to learn effective policies. Therefore, we propose the Module-based Maskable PPO (MMPPO) to improve the action selection process in the policy gradient algorithm.

In our algorithm, for one module, its masked actions are as follows:

- If no package is located on the module, all movement actions of the module are prohibited.
- If there is one package on the module, and the module is positioned at the edge of the platform, actions that could result in the package exceeding the platform boundaries are also prohibited.
- Collisions are not regarded as invalid actions.

During the training process, actions are sampled only from valid actions. This method ensures more stable training compared to giving negative rewards to invalid actions.

Algorithm 1 shows the pseudo-code of MMPPO on Omniveyor. The loss function $J_{ppo}(\theta)$ for updating and optimizing the weight parameters of $\pi(a_t|s_t;\theta_{now})$ is given by

$$J^{\theta_k}_{ppo}(\theta) \approx \sum_{(s_t,a_t)} \min\big(r_t(\theta)A^{\pi_{\theta_k}}(s_t,a_t), \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)A^{\pi_{\theta_k}}(s_t,a_t)\big), \tag{14}$$

where the $r_t(\theta)$ means

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, \tag{15}$$

and $A^{\pi_{\theta_k}}(s_t, a_t)$ is the estimator of advantage function. θ is the new policy and θ_k is the old policy. $\text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)$

Algorithm 1 MMPPO (Module-Based Maskable PPO)

Input: Global information about the platform
Output: Actions of the modules
Initial policy parameters θ_0 and generate a simulated logistics environment for training
for *iteration* = *1,2,...,N* **do**
  Reset the simulated logistics environment
  Mask the invalid actions
  for *actor* = *1,2,...,T* **do**
    Observe the package state s_t of the environment
    Run policy $\pi(\cdot|s_t;\theta)$ to choose action a_t based on the environment state s_t
    After the action a_t is executed, the environment feeds back a reward value r_t to the system and converts to a new state s_{t+1}
  end
  Compute the value of advantage function $A^{\pi_{\theta_k}}(s_t, a_t)$
  Find θ optimizing the loss function $J^{\theta_k}_{ppo}(\theta)$
  Replace the parameters of actor network $\theta_k \leftarrow \theta$
end

TABLE II
REWARD STRUCTURE FOR PLATFORM

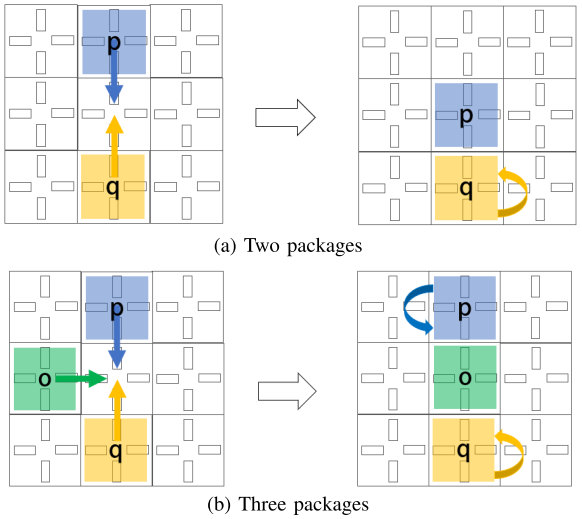
Action	Reward
Push (far away from the destination)	-0.4
Push (towards to the destination)	-0.3
Wait (when package on module)	-0.5
Package Collision	-5
Finish Episode	30

ensures that $r(\theta)$ can only be located in the interval $(1-\epsilon, 1+\epsilon)$. ϵ is a hyper-parameter that controls how far the new policy is allowed to deviate from the old policy.

C. Package Following Problem

In the simulation environment of reinforcement learning-based MAPF, the package following problem is tricky, which makes the algorithm performance deviate from the theoretical results. Stern et al. [39] first point out this problem, which means that one agent plans to occupy a vertex occupied by another agent in the previous time step. Sartoretti et al. [34] further emphasize that this problem implies that planners prohibit agents from following each other without any space to find shorter paths. In the simulation, when two adjacent packages move in the same direction, the trailing package must wait until the leading one has successfully moved to the next module before it can proceed. This can potentially increase makespan due to waiting. However, since the speed of all modules is consistent, packages can follow each other on the platform. Therefore, to minimize makespan and increase throughput more effectively, we experiment with the following three ways in our simulation:

1) **MMPPO-Restart**: end the training episode and restart a new episode after each collision.
2) **MMPPO-Loop**: avoid collisions by controlling through a loop. The actions of modules are executed sequentially after sorting the packages, ensuring that the next move of each package will not collide with others. Subsequently, the positions of the packages are updated.



(a) Two packages

(b) Three packages

Fig. 6. The movement of two or three packages if the vertex conflict happens.

3) **MMPPO-Matrix**: calculate the next positions of all packages based on the results of the algorithm with matrix multiplication, and then make adjustments according to the positions of the packages. If a swapping conflict occurs, the corresponding packages are returned to their original location. If a vertex conflict occurs, a randomly selected package is returned. The process is repeated until no collisions exist.
As shown in Fig. 6 (a), when two packages attempt to enter the same module at time t, package p moves forward while package q stays. At time $t+1$, scheduling continues based on their new states. And in Fig. 6 (b), if three packages attempt to enter the same platform, package o moves to the central module, while packages p and q remain in place. Scheduling for all three resumes at time $t+1$, based on their updated states and goals.

Fig. 7 shows the efficiency of our implementation. The blue dashed line represents the number of steps CBS requires to complete the task, serving as the optimal solution [28]. If we simulate our algorithm in the first way that ends the

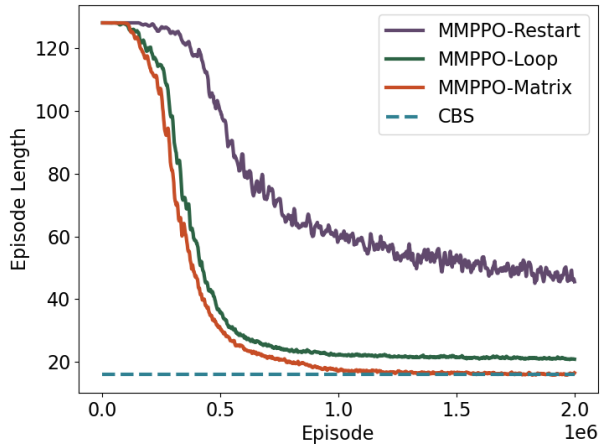


Fig. 7. Mean episode length during training, which indicates the time required for the platform to deliver all packages to their destinations, lower is better.

training episode directly (purple line), the convergence speed is slow, and satisfactory results cannot be achieved. Although the simulation in the second way (green line) converges successfully, there is an unexpected gap between the result and the optimal one. On the contrary, the third way (orange line) is more efficient and can achieve the optimal result.

V. Experimental Results and Prototype Implementation

In this section, we present the simulation results comparing our algorithm with several MAPF solvers in the grid world. We conducted experiments in static and dynamic pipeline environments. The static environment refers to transporting a fixed number of packages to their destination, while the dynamic environment refers to the continuous input and delivery of packages on the platform. These experiments comprehensively compare the performance of algorithms in environments with different grid sizes and numbers of packages. In addition, we also conducted a performance comparison between the square platform and the hexagonal platform. All experiments are conducted on a computer equipped with AMD EPYC 7402 24-Core Processor and NVIDIA GeForce RTX 3090. The version of Python used is 3.9.18. Furthermore, we build a prototype of Omniveyor with omni-wheel modules.

A. Experiments in Static Environment

Since few algorithms combine modules and packages uniformly based on SCS, we employ two representative methods for solving MAPF problem as baselines, Conflict-Based Search (CBS) and Prioritized Planning Algorithm (PPA).

- CBS is the optimal solver with the A* algorithm and conflict tree.
- PPA is a solver where each package is assigned a priority, and the trajectory for each package is planned using the A* algorithm.

Both of them are centralized planners and have been widely applied in the industry field. After scheduling the path toward these planners, we allocate the path of the package to each module to achieve the same effect as our algorithm. Please refer to the Appendix I for detailed algorithms.

For MMPPO, we employ Actor-Critic policy as the policy network with an initial learning rate of 0.0003. Moreover, as the platform expands, we gradually increase time steps and max cycles for each episode. World sizes are 5×5, 7×7, 8×8, 10×10, corresponding to module numbers 25, 49, 64, 100, respectively. We design two scenarios: one for moving packages from specified starting points to endpoints. For each world size, we have several different package numbers: 3, 5 for 25 modules, 3, 5, 10 for 49 and 64 modules, and 3, 5, 10, 15 for 100 modules, respectively. The other scenario involves randomly placing packages anywhere on the platform except the endpoint and transporting them. For this scenario, we use 10 package numbers for 25 modules, 15 for 49 and 64 modules, and 20 for 100 modules.

In addition to makespan and cost, we also recorded runtime, which includes both computation and execution time. Given the nature of reinforcement learning, the MMPPO algorithm integrates platform control with path planning, leading to synchronized computation and execution. In contrast, traditional MAPF algorithms, such as CBS and PPA, require additional decomposition of paths, where each step must be assigned to individual platform controllers. Thus, directly comparing computation time alone would be misleading. However, since all algorithms are executed on the same simulation platform, comparing overall runtime remains valid.

According to the experimental data in Table III, our algorithm demonstrates performance comparable to CBS or close to optimal on sparse or smaller platforms, effectively achieving optimal results. When the package density is low relative to the platform size, our algorithm shows no significant advantage in makespan but outperforms PPA in terms of total path cost. As the number and density of packages increase, the CBS algorithm struggles to effectively plan paths within the specified time constraints. This challenge arises from the need for multiple packages to reach the same destination in the Omniveyor system, which significantly escalates the complexity of the CBS algorithm. Consequently, our comparisons are limited to MMPPO and PPA. Clearly, on small platforms with high package density, our algorithm performs better than PPA in both path cost and makespan. In terms of runtime, our algorithm shows a clear advantage on smaller platforms, as its tighter integration with the platform eliminates the additional steps required in traditional MAPF to translate paths into control commands.

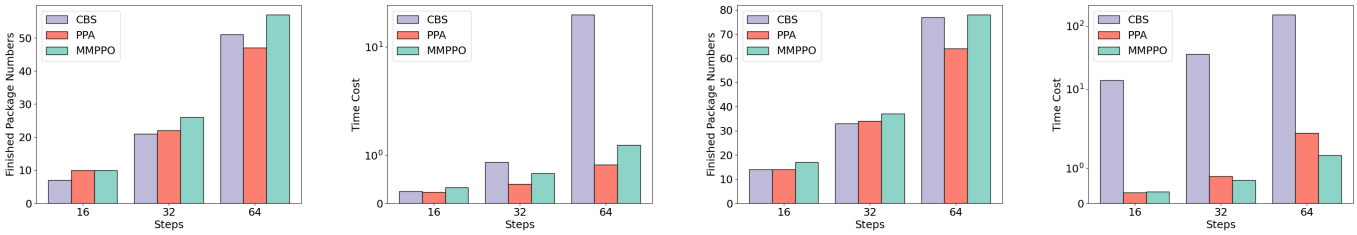
It should be noted that due to different random seeds, the entry and exit locations of packages vary across test environments. This explains why the CBS algorithm was unable to find a valid solution for 10 packages on the 7×7 platform, while all three algorithms were able to find solutions on the 8×8 platform. For the 10×10 platform, although increasing the number of packages from 10 to 15 did not significantly optimize makespan, the cost advantage improved markedly with higher package density.

Although MMPPO does not show a significant advantage in runtime compared to PPA at high package densities in this experiment, it is important to note that *in real-world scenarios, packages continuously enter the system*. This results in dynamic changes to the number of packages, their starting

TABLE III

COMPARISON OF PPA, CBS, AND MMPPO UNDER NON-ASSEMBLY CONDITIONS, "−" INDICATES THAT THE ALGORITHM EXCEEDS THE UPPER LIMIT OF THE DESIGN TIME FOR A SINGLE RUN. THE NUMBERS REPRESENT THE SIZES OF MAKESPAN, PATH COSTS, AND TIME COSTS(MS) IN DIFFERENT ENVIRONMENTS

Parameters		CBS			PPA			MMPPO		
Size	N	Cost	Makespan	Time↓	Cost	Makespan	Time↓	Cost	Makespan	Time↓
5×5	3	**12**	5	197	14	5	192	**12**	5	**36**
5×5	5	**23**	**5**	199	26	6	206	**23**	**5**	**36**
5×5	10	-	-	1000	79	10	208	**59**	**9**	**63**
7×7	3	**20**	7	200	23	7	203	21	7	**143**
7×7	5	**44**	11	203	57	11	205	47	11	**173**
7×7	10	-	-	2000	90	13	218	**86**	**12**	**197**
7×7	15	-	-	2000	159	16	**231**	**144**	16	262
8×8	3	24	9	206	24	9	204	24	9	**130**
8×8	5	**45**	11	237	50	11	213	**45**	11	**192**
8×8	10	**89**	**10**	629	104	11	236	**89**	11	**220**
8×8	15	-	-	2000	154	**12**	**256**	**135**	14	286
10×10	3	**23**	10	**205**	26	10	219	**23**	10	389
10×10	5	**64**	16	**214**	72	16	224	**64**	16	452
10×10	10	-	-	2000	153	**19**	**252**	**147**	20	497
10×10	15	-	-	2000	225	**18**	**270**	**213**	22	582
10×10	20	-	-	2000	347	**22**	**425**	**278**	27	887



(a) Efficiency on sparse input (b) Time on sparse input (c) Efficiency on dense input (d) Time on dense input

Fig. 8. The efficiency (finished package numbers) and time cost on assembly operations of the three algorithms: CBS, PPA, and MMPPO under different input densities. For efficiency, higher is better; for time cost, lower is better.

points, and endpoints at each moment. Consequently, PPA re-evaluates its path planning each time a new package arrives, leading to a notable increase in computation time. In contrast, our algorithm MMPPO employs stepwise reasoning, which enhances its performance under these conditions, as demonstrated in the dynamic environment experiments in the next section.

It is apparent that when operating in a static environment, our algorithm can achieve or even outperform traditional path planning algorithms.

B. Experiments in Dynamic Pipeline Environment

In the simulated dynamic pipeline environment, we evaluated the performance of CBS, PPA, and MMPPO on different module platforms with varying densities. During the evaluation, we recorded the stability, efficiency, and time consumption generated by each algorithm. We use total finished package numbers as efficiency.

In a dynamic environment, as long as there are entrances at any given step, packages can enter the platform in a pipeline manner. In the real sorting scenarios, the number of packages entering the platform follows a Poisson process. The expected rate of packages entering the platform from each entrance within a fixed period could be calculated from the Poisson formula:

$$f(X = k) = \frac{\mu^k e^{-\mu}}{k!}, k = 0, 1, 2, \ldots$$

The situation with different package densities can be simulated by adjusting the parameter μ.

1) Stability: We evaluated the stability of different algorithms under various densities. To ensure pipeline efficiency, we restricted the calculation time of each algorithm to a relatively appropriate duration for different size worlds (10 minutes for 10×10 size) and experimented with 64 steps. In traditional path planning algorithms, whenever a new package enters, all packages on the platform will be re-planned. However, in our proposed model, the addition of new packages does not require any additional processing. The model can handle the addition of new packages adaptively.

If the package numbers remain stagnant within 8 steps and the final count does not increase, we identify this as a platform blockage, indicating a lack of stability. We conducted 100 tests for each environment and the experimental results are detailed in Table IV. With the increase in platform size, the stability

TABLE IV
THE STABILITY COMPARISON OF PPA, CBS, AND MMPPO IN THE DYNAMIC ENVIRONMENT, HIGHER IS BETTER

size	algorithm	μ=0.5	μ=1	μ=1.5
5×5	CBS	100%	92%	**73%**
5×5	PPA	100%	90%	54%
5×5	MMPPO	100%	**93%**	67%
size	algorithm	μ=0.5	μ=1	μ=2
7×7	CBS	99%	90%	64%
7×7	PPA	**100%**	98%	68%
7×7	MMPPO	**100%**	**100%**	**85%**
size	algorithm	μ=0.5	μ=1	μ=2
10×10	CBS	95%	87%	33%
10×10	PPA	**100%**	**100%**	97%
10×10	MMPPO	**100%**	**100%**	**99%**

of PPA and MMPPO has improved under the same input, and MMPPO has significant advantages in high-density situations. The reason is that in high-density situations, CBS is unable to calculate the required paths on time, while PPA is prone to situations where no solution is found. Additionally, we assessed package coverage rates With μ=0.5, μ=1, μ=1.5 density in a 7×7 platform, the max package coverage can reach 16.3%, 40.8%, 71.4%, implying that there will be 35 packages on the platform. This signifies a relatively dense situation.

2) Efficiency and Time Cost: Due to the observed instability of the MAPF algorithm, we selected relatively average solutions as benchmark data to guarantee fair and accurate comparisons. In Fig. 8 we selected two different input densities with the average rate of incoming packages follows Poisson process at each entrance as 1 (i.e., μ=1, the sparse scenario shown in Fig. 8(a) and Fig. 8(b)) or 2 (i.e., μ=2, the dense scenario shown in Fig. 8(c) and Fig. 8(d)) in 7×7 size world.

During the experiment, we set three observation steps 16, 32, and 64. In each algorithm, the initial number of packages on the platform is uniformly set to 3. As shown in Fig. 8(a) and Fig. 8(c), when the step size is small, there are no significant differences among the three algorithms with different efficiency or densities. However, as the step size gradually increases, it becomes evident that the efficiency of CBS and our algorithm significantly exceeds PPA. This is primarily because the former two algorithms consistently aim to compute the shortest paths for all packages on the platform. However, as shown in Fig. 8(b) and Fig. 8(d), when considering the aspect of time cost, CBS requires significantly more time than PPA and our algorithm. Specifically, as the step size increases linearly, the time cost of CBS increases exponentially, making it unsuitable for practical assembly applications. In contrast, the algorithm we proposed is equivalent to PPA in performance and shows certain advantages in high-density environments.

Furthermore, we illustrate the cumulative number of delivered packages by PPA and our MMPPO on Omniveyor in Fig. 9. Generally speaking, as the time step goes on, the number continues to grow. MMPPO consistently outperforms PPA across 2 scenarios. However, in high-density environments,

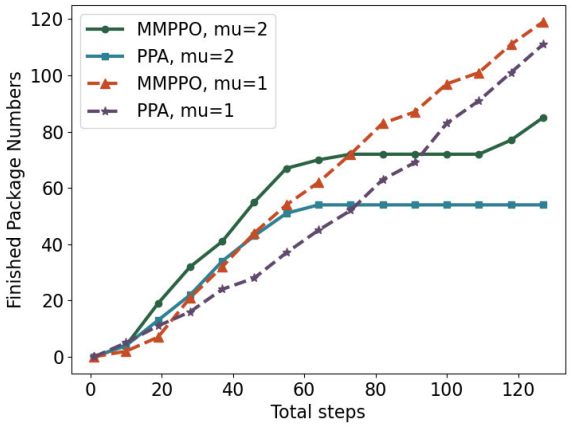


Fig. 9. Comparison of the cumulative number of delivered packages by PPA and MMPPO under two different density inputs.

the growth rate of both algorithms gradually slows down as the step size increases. This is primarily due to the number of packages on the platform gradually becoming saturated, making it difficult to move packages. In addition, the PPA system will be unable to continue transporting packages in high-density environments, but MMPPO can resume sorting after a while. Conversely, under low-density situations, the efficiency can maintain a linear growth trend, which has better performance than high-density in larger steps.

C. Comparison With Hexagonal SCS

We further conduct comparative experiments on the hexagonal SCS in both static and dynamic environments. We train the MMPPO algorithm on a simulation platform based on Celluveyor, and test in 5×5, 7×7, 10×10 grids, with 3, 5, and 10 packages. Two scenarios were evaluated: one involving path planning from fixed starting points to designated endpoints, and another where packages were randomly placed on the platform.

In the static environment, where the start and end points of the packages were fixed, we compared the difficulty of path planning. The results including path cost, makespan, and runtime, are shown in Table V. To better emphasize the performance differences between the platforms, we used a more complex experimental setup than that presented in Table III.

The results demonstrate that the square platform outperforms the hexagonal one. We observe that on smaller platforms like the 5×5 grid, the performance of the hexagonal platform is comparable to that of the square platform. However, as the grid size increased to 10×10, the computational speed of the hexagonal platform declined significantly due to increased path-planning complexity, causing performance degradation and making it less efficient than the square platform.

Additionally, we evaluate the dynamic performance of the platforms by introducing packages according to a Poisson distribution, using the same expected values to compare throughput across different platforms. Due to the limitations of three-directional movement, the comparison was made between the square platform and the hexagonal platform with six-directional movement. The results of these experiments are presented in Figure 10.

TABLE V

COMPARISON OF PERFORMANCE BETWEEN SQUARE AND HEXAGONAL PLATFORMS. FOR ALL DATA POINTS, LOWER VALUES INDICATE BETTER PERFORMANCE. IT IS EVIDENT THAT THE SQUARE PLATFORM EXHIBITS CERTAIN PERFORMANCE ADVANTAGES

Parameters		Square			Hexagon		
Size	N	Cost	Makespan	Time↓	Cost	Makespan	Time↓
5×5	3	**12**	**5**	**36**	13	6	56
5×5	5	**22**	7	**65**	23	7	73
5×5	10	**70**	**11**	**105**	85	17	162
7×7	3	**21**	**7**	**143**	24	12	193
7×7	5	47	11	**173**	**39**	**10**	191
7×7	10	**86**	**12**	**197**	86	19	351
10×10	3	**23**	**10**	**389**	47	28	1022
10×10	5	**64**	**16**	**452**	114	33	1162
10×10	10	**147**	**20**	**497**	159	33	1328

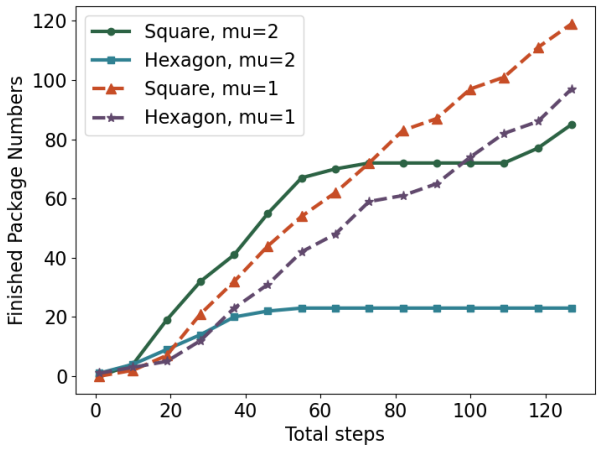


Fig. 10. Comparison of the cumulative number of delivered packages by square and hexagonal platform under two different density inputs.

When the number of packages is small, the square platform demonstrates a noticeable performance advantage over the hexagonal platform. As package density increases, this advantage becomes more pronounced. This is because higher package densities increase the complexity of the hexagonal platform, causing it to reach saturation more quickly, which negatively impacts its overall throughput performance.

D. Omniveyor Implementation

We design and develop a prototype of the Omniveyor system, utilizing modules equipped with omni-wheels for efficient package sorting. Furthermore, we construct a 5×5 Omniveyor platform and successfully apply it to a book sorting application.

Fig. 11(a) shows the structure of the module. The driving device comprises a motor equipped with a Hall encoder. The motors of each transmission module are uniformly controlled by the STM32 driver board. Multiple driver boards are centrally controlled by the Raspberry Pi, which in turn communicates with the server. Control instructions are issued and fed back between the Raspberry Pi and the server through wireless LAN. The Omniveyor we built is shown in Fig. 11(b) and Fig. 11(c), which is a sorting platform composed of multiple transmission modules with scalable capabilities.

(a) Top view of the module (b) 3×3 grid platform (c) Book sorting system with 5×5 Omniveyor platform

Fig. 11. The physical implementation and deployment of Omniveyor.

The control process following package entry onto the platform is as follows:

1) **Initialization**. The server configures the environment based on the existing platform size. After the sensor determines the initial position of the package, the information will relayed back to the server.
2) **Controlling**. After obtaining the starting and ending points of all packages, we use MMPPO to plan the controller, calculating the control instructions of each module at specified times. The server uniformly dispatches the instructions to each Raspberry Pi, subsequently relayed by the driver boards to the designated motors.
3) **Planning**. During the sorting process, the visual positioning module continuously monitors the position of each package in real time. In instances where multiple packages have path conflicts or collisions within the same module due to control errors, we will re-plan the control instructions immediately.

Our algorithm could accomplish the task of continuously delivering packages to designated locations on the simplified platform we constructed.

VI. CONCLUSION

In this paper, we design Omniveyor, a scalable logistics sorting system constructed with the omni-wheel platform, and present a reinforcement learning-based control algorithm for scheduling. Different from traditional path planning methods, our study concentrates on directly controlling the modules to optimize the package sorting process. We establish the mathematical model and simulation environment, and employ

MMPPO to effectively control the modules. Comparative simulations show that in a static environment, our algorithm performs comparably to CBS. Moreover, it achieves higher throughput than PPA in the dynamic pipeline environment where packages are entered continuously.

With the platform scale expansion, the training time and accuracy of reinforcement learning will become challenges. Therefore, subsequent research will focus on exploring multi-agent reinforcement learning with local observation to further improve the system performance. Additionally, we plan to refine our mechanical structure to better meet industrial requirements.

APPENDIX A
COMPARISON ALGORITHM INTRODUCTION

PPA is a planning algorithm often used in practice applications [40]. Packages are prioritized in decreasing order of priority to avoid collisions with static obstacles and previous packages [41], [42]. Algorithm 2 provides the pseudo-code in Omniveyor, with priorities determined based on the sequence in which packages enter the platform.

Algorithm 2 Prioritized Planning Algorithm

Input: Starts and destinations of packages
Output: Actions of the modules
Sort packages as $b_1, \ldots, b_n$, in which b_1 has the highest priority.
Use A* algorithm to compute a collision-free path τ_1, and compute a timing function σ_1.
for $i \leftarrow 2$ **to** n **do**
  for $j \leftarrow 1$ **to** $i-1$ **do**
    Formulate the packages as moving obstacles $b_j(\tau_j \circ \sigma_j) \subset \mathcal{O}(t)$.
  end
  Use A* algorithm to compute a collision-free path τ_i, and timing function σ_i with obstacle set $\mathcal{O}(t)$.
end
for $k \leftarrow 1$ **to** m **do**
  Calculate actions of the modules with τ_k and σ_k.
end

CBS is a well-known MAPF solver that typically has an optimal solution when the number of packages is small [28]. The primary concept of the algorithm involves dividing planning into two levels. At the low level, the traditional A* algorithm is used to carry out single-agent planning with constraints. Meanwhile, at the high level, the algorithm uses a conflict tree to select and resolve collisions. In instances where conflicts are detected, constraints are applied to the low level, prompting it to replan the path until conflicts are eliminated.

To accommodate the fact that Omniveyor allows multiple packages to share the same exit, meaning that different packages can arrive at the same target vertex v at different times t, we introduced modifications to adapt CBS to our platform. In the MAPF framework, this behavior implies that agents are immediately removed from the platform upon reaching their goal. To handle this, we incorporate a virtual node $(-1, -1)$ at the end of the path of each package to manage its removal. The constraints for any package are now defined in two scenarios:

1) If $v \neq (-1, -1)$ in a conflict tuple (b_i, b_j, v, t), the original collision rule applies: agents b_i and b_j cannot occupy the same vertex v at the same time t.
2) If $v = (-1, -1)$, no restrictions apply at any time t, meaning agents a_i and a_j can occupy the virtual node simultaneously. In other words, conflicts at the virtual node $(-1, -1)$ are not considered conflicts.

The modified high-level CBS structure is illustrated in Algorithm 3, where SIC represents the sum of individual costs heuristic. Regarding the experimental setup, the data in Section V-A replicates CBS using the implementation based on an open-source repository [43].

Algorithm 3 High-Level of CBS for Omniveyor

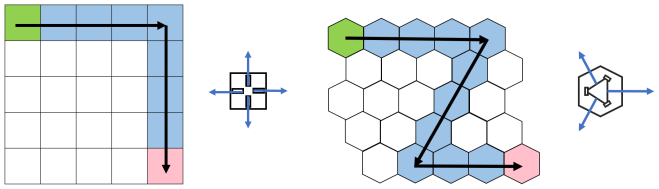
Input: Starts and destinations of packages
Output: Actions of the modules
$Root.constraints = \emptyset$
$Root.solution$ = find individual paths using the A*(), appending with (-1,-1)
$Root.cost = SIC(Root.solution)$
insert Root to OPEN
while *OPEN not empty* **do**
  P ← best node from OPEN
  Validate the paths in P until a conflict occurs.
  if *P has no conflict* **then**
    return *P.solution*
  end
  C ← first conflict (b_i, b_j,v, t) in P and (-1,-1) not in v
  foreach *package* b_i *in* C **do**
    A ← new node
    A.constraints ← P.constraints + (b_i, s, t)
    A.solution ← P.solution
    Update A.solution by involving A*(b_i)
    $A.cost = SIC(A.solution)$
    Insert A to OPEN
  end
end

APPENDIX B
COMPARISON OF QUADRILATERAL AND HEXAGONAL PLATFORMS

The advantages of the square-module-based Omniveyor platform compared to the hexagonal-module-based Celluveyor are as follows:

A. Motion Control and Efficiency

The square design offers certain advantages in control, especially in tightly arranged SCS systems, where smaller packages are easier to handle [2]. We assume that each package entirely covers a controller, with each controller serving a single package. In the square configuration of Omniveyor, packages can move in four directions. Although packages



(a) Square module, four directions to move

(b) Hexagonal module, movement is restricted to three directions by the controller

Fig. 12. The theoretical shortest path from the same starting point to the endpoint shows that the hexagonal design results in a longer shortest path than the square design.

on the Celluveyor are theoretically capable of moving in six directions [44], practical constraints limit this module to changing direction at a minimum angle of 120°, as depicted by the three directions [45]. The insufficient driving force provided by the omnidirectional wheels renders movement in the other three directions impractical, thereby significantly complicating control.

B. Path Planning and Practical Application

Based on the analysis presented above, when package movement is restricted to three directions, as shown in Figure 12, a platform composed of 25 modules arranged in a 5×5 grid exhibits a longer shortest theoretical path from the starting point (marked in green) to the endpoint (marked in red) compared to a square design. Specifically, the path length for the three-direction platform is 11 steps, while it is only 8 steps for the square configuration. Furthermore, in high-density SCS environments, the Celluveyor tends to transport large packages along straight lines. These packages span multiple modules, despite their omnidirectional movement capabilities. As a result, the square layout proves to be more logical and efficient under these operational conditions.

C. Improved Space Utilization and Reduced Costs

The square design allows for more efficient space utilization, particularly in traditional factory settings where workstations are typically arranged in straight lines. Square modules can be packed more tightly, facilitating easier expansion and modification of the system. In contrast, the boundary design of the Celluveyor may necessitate the disassembly of individual modules. For example, adapting the platform for warehouse use may require cutting certain modules and positioning single wheels at the edges. This modular control approach diminishes overall control efficiency.

Furthermore, a comparative analysis of the structural characteristics of the two omnidirectional platforms, as discussed in reference [45], evaluated the cost and effectiveness of square versus hexagonal designs. The study compared the number of wheels needed to cover the same area, the number of control boards required, the number of motors necessary to transport the same distance, and the factors contributing to operational failures. The experiment results reveal that the hexagonal structure experienced more operational issues during testing, such as packages slipping in place and heavier packages failing to move to adjacent cells. In addition, the square design emerged as superior in terms of both cost-effectiveness and control efficiency.

REFERENCES

[1] Y. Jaghbeer, R. Hanson, and M. I. Johansson, "Automated order picking systems and the links between design and performance: A systematic literature review," *Int. J. Prod. Res.*, vol. 58, no. 15, pp. 4489–4505, Aug. 2020.

[2] J. S. Keek, S. L. Loh, and S. H. Chong, "Design and control system setup of an E-pattern omniwheeled cellular conveyor," *Machines*, vol. 9, no. 2, p. 43, Feb. 2021.

[3] L. Zhen and H. Li, "A literature review of smart warehouse operations management," *Frontiers Eng. Manage.*, vol. 9, no. 1, pp. 31–55, Mar. 2022.

[4] G. Fragapane, R. de Koster, F. Sgarbossa, and J. O. Strandhagen, "Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda," *Eur. J. Oper. Res.*, vol. 294, no. 2, pp. 405–426, Oct. 2021.

[5] Z. Seibold, K. Furmans, and K. R. Gue, "Using logical time to ensure liveness in material handling systems with decentralized control," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 1, pp. 545–552, Jan. 2022.

[6] C. Uriarte, H. Thamer, and M. Freitag, "Celluveyor–Omnidirektionale zellulare Fördertechnik," *Industrie Management*, vol. 31, no. 6, pp. 20–23, 2015.

[7] C. Uriarte, H. Thamer, M. Freitag, and K. Thoben, "Flexible automatisierung logistischer prozesse durch modulare roboter," *Logistics J., Proc.*, vol. 2016, no. 5, pp. 1–6, May 2016.

[8] W. Zaher, A. W. Youssef, L. A. Shihata, E. Azab, and M. Mashaly, "Omnidirectional-wheel conveyor path planning and sorting using reinforcement learning algorithms," *IEEE Access*, vol. 10, pp. 27945–27959, 2022.

[9] T. Kruhn, M. Radosavac, N. Shchekutin, and L. Overmeyer, "Decentralized and dynamic routing for a cognitive conveyor," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mechatronics*, Jul. 2013, pp. 436–441.

[10] J. Fleischmann and K. Furmans, "Sequencing with decentralized control using modular highest-density conveyor systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3001–3011, Jul. 2024.

[11] S. Lin, A. Liu, J. Wang, and X. Kong, "A review of path-planning approaches for multiple mobile robots," *Machines*, vol. 10, no. 9, p. 773, 2022.

[12] H. Hu, X. Yang, S. Xiao, and F. Wang, "Anti-conflict AGV path planning in automated container terminals based on multi-agent reinforcement learning," *Int. J. Prod. Res.*, vol. 61, no. 1, pp. 65–80, Jan. 2023.

[13] J.-Y. Jang, S.-J. Yoon, and C.-H. Lin, "Automated guided vehicle (AGV) driving system using vision sensor and color code," *Electronics*, vol. 12, no. 6, p. 1415, Mar. 2023.

[14] R. Stefanini and G. Vignali, "Environmental and economic sustainability assessment of an Industry 4.0 application: The AGV implementation in a food industry," *Int. J. Adv. Manuf. Technol.*, vol. 120, nos. 5–6, pp. 2937–2959, May 2022.

[15] T. Kruhn, S. Falkenberg, and L. Overmeyer, "Decentralized control for small-scaled conveyor modules with cellular automata," in *Proc. IEEE Int. Conf. Autom. Logistics*, Aug. 2010, pp. 237–242.

[16] K. R. Gue, K. Furmans, Z. Seibold, and O. Uludag, "GridStore: A puzzle-based storage system with decentralized control," *IEEE Trans. Autom. Sci. Eng.*, vol. 11, no. 2, pp. 429–438, Apr. 2014.

[17] Z. Seibold, T. Stoll, and K. Furmans, "Layout-optimized sorting of goods with decentralized controlled conveying modules," in *Proc. IEEE Int. Syst. Conf. (SysCon)*, Apr. 2013, pp. 628–633.

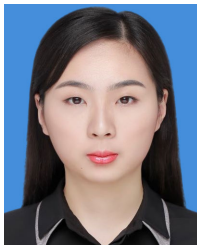
[18] K. Wang, W. Liang, H. Shi, J. Zhang, and Q. Wang, "Driving line-based two-stage path planning in the AGV sorting system," *Robot. Auto. Syst.*, vol. 169, Nov. 2023, Art. no. 104505.

[19] Z. Xing, H. Yue, T. Zhang, W. Wu, and R. Hu, "Collision avoidance and give way of heterogeneous and variable-sized multiple mobile robots based on glued nodes," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3627–3638, Jul. 2024.

[20] S. Lin, A. Liu, J. Wang, and X. Kong, "An improved fault-tolerant cultural-PSO with probability for multi-AGV path planning," *Expert Syst. Appl.*, vol. 237, Mar. 2024, Art. no. 121510.

[21] B. Kang, C. Park, H. Kim, and S. Hong, "Bayesian optimization for the vehicle dwelling policy in a semiconductor wafer fab," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 4, pp. 5942–5952, Oct. 2024.

[22] A. S. Bhadoriya, C. M. Montez, S. Rathinam, S. Darbha, D. W. Casbeer, and S. G. Manyam, "Optimal path planning for a convoy-support vehicle pair through a repairable network," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 4, pp. 4936–4947, Oct. 2024.
[23] D. Xiang, H. Lin, J. Ouyang, and D. Huang, "Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot," *Sci. Rep.*, vol. 12, no. 1, p. 13273, Aug. 2022.
[24] J. Li et al., "Lifelong multi-agent path finding in large-scale warehouses," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11272–11281.
[25] J. Cartucho, R. Ventura, and M. Veloso, "Robust object recognition through symbiotic deep learning in mobile robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 2336–2341.
[26] C. Huang, Z. Ming, and H. Huang, "Drone stations-aided beyond-battery-lifetime flight planning for parcel delivery," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 4, pp. 2294–2304, Oct. 2023.
[27] J. Yu and S. M. LaValle, "Structure and intractability of optimal multi-robot path planning on graphs," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2013, vol. 27, no. 1, pp. 1443–1449.
[28] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artif. Intell.*, vol. 219, pp. 40–66, Feb. 2015.
[29] S. D. Han and J. Yu, "DDM: Fast near-optimal multi-robot path planning using diversified-path and optimal sub-problem solution database heuristics," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1350–1357, Apr. 2020.
[30] A. Skrynnik, A. Andreychuk, M. V. Nesterova, K. Yakovlev, and A. I. Panov, "Learn to follow: Decentralized lifelong multi-agent pathfinding via planning and learning," in *Proc. AAAI Conf. Artif. Intell.*, Jan. 2023, pp. 17541–17549.
[31] J. Hu, H. Niu, J. Carrasco, B. Lennox, and F. Arvin, "Voronoi-based multi-robot autonomous exploration in unknown environments via deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14413–14423, Oct. 2020.
[32] Y. Guan, S. Zou, H. Peng, W. Ni, Y. Sun, and H. Gao, "Cooperative UAV trajectory design for disaster area emergency communications: A multiagent PPO method," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8848–8859, Mar. 2024.
[33] T. M. Ho, K.-K. Nguyen, and M. Cheriet, "Federated deep reinforcement learning for task scheduling in heterogeneous autonomous robotic system," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 528–540, Nov. 2022.
[34] G. Sartoretti et al., "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2378–2385, Jul. 2019.
[35] T. Zhang et al., "Multi-agent collaboration via reward attribution decomposition," 2020, *arXiv:2010.08531*.
[36] K. Okumura, M. Machida, X. Défago, and Y. Tamura, "Priority inheritance with backtracking for iterative multi-agent path finding," *Artif. Intell.*, vol. 310, Sep. 2022, Art. no. 103752.
[37] J. Banfi, N. Basilico, and F. Amigoni, "Intractability of time-optimal multirobot path planning on 2D grid graphs with holes," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 1941–1947, Oct. 2017.
[38] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," 2020, *arXiv:2006.14171*.
[39] R. Stern et al., "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proc. 12th Annu. Symp. Combinat. Search (SoCS)*, 2019, vol. 10, no. 1, pp. 151–158.
[40] M. Cap, P. Novak, A. Kleiner, and M. Selecky, "Prioritized planning algorithms for trajectory coordination of multiple mobile robots," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, pp. 835–849, Jul. 2015.
[41] J. P. van den Berg and M. H. Overmars, "Prioritized motion planning for multiple robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Aug. 2005, pp. 430–435.
[42] S. LaValle, *Planning Algorithms*, vol. 2. Cambridge, U.K.: Cambridge Univ. Press, 2006, pp. 3671–3678.
[43] Msaudulhassan. (2020). *Mapf: Python Implementation of Space-time A*, Prioritized Planning, Conflict-based Search for Multi-Agent Path Finding*. [Online]. Available: https://github.com/msaudulhassan/mapf
[44] M. B. Firvida, H. Thamer, C. Uriarte, and M. Freitag, "Decentralized omnidirectional route planning and reservation for highly flexible material flow systems with small-scaled conveyor modules," in *Proc. IEEE 23rd Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2018, pp. 685–692.
[45] T. Yu and J. Liu, "Intelligent logistics desktop based on target detection and package reidentification," M.S. thesis, Fac. Comput., Harbin Inst. Technol., Harbin, China, 2023.



Mingrui Yin received the master's degree in information management from the University of Illinois at Urbana–Champaign in 2019. She is currently pursuing the Ph.D. degree in computer science and technology with Harbin Institute of Technology, Harbin, China. Her research interests include logistics systems and path planning.

Hao Zhang received the Ph.D. degree from the University of Science and Technology of China in 2014. He is currently an Associate Researcher with the Computer Science Department, Harbin Institute of Technology (HIT), China. His research interests include deep learning application, federated learning, and pervasive computing.

Chenxin Cai received the master's degree in computer technology from Harbin Institute of Technology, Harbin, China, in 2022, where she is currently pursuing the Ph.D. degree in computer science and technology. Her research interests include multi-robot systems and path planning.

Meiyan Liang received the bachelor's degree in computer science and technology from Harbin Institute of Technology, Harbin, China, in 2024, where she is currently pursuing the master's degree in electronic information. Her research interests include drive control algorithm and systems.

Jie Liu (Fellow, IEEE) is currently a Chair Professor with Harbin Institute of Technology (HIT), China, and the Dean of the AI Research Institute. He is an ACM Distinguished Scientist. Before joining HIT, he spent 18 years with Xerox PARC, Microsoft Research, and Microsoft Product Teams. As a Principal Research Manager with MSR, he led the Sensing and Energy Research Group (SERG). In MSR-NExT and product groups, he incubated smart retail solutions, which became part of Microsoft Business AI offering. His research interests include root in understanding and managing the physical properties of computing. He also has chaired a number of top-tier conferences in sensing and pervasive computing and was an associate editor of several top-tier journals.